

Módulo 7 – Capítulo 06 – Sección 01

Por qué multiagente: especialización, paralelismo y reducción del contexto por agente

El capítulo anterior estableció cuatro frameworks para construir agentes individuales. Este capítulo examina cuándo y cómo combinar múltiples agentes en sistemas coordinados. La pregunta de partida no es "¿cómo construir un sistema multiagente?" sino "¿cuándo es un sistema multiagente la solución correcta?". La respuesta incorrecta —y frecuente— es "cuando la tarea es compleja". La respuesta correcta es más específica: cuando la tarea supera la capacidad de un agente individual en alguna dimensión concreta y medible, y cuando el beneficio de esa extensión supera el costo de la coordinación adicional.

Los sistemas multiagente no son una extensión caprichosa de los agentes individuales: responden a limitaciones concretas que se hacen visibles cuando se intenta usar un agente único para tareas que superan ciertos umbrales. La primera limitación es la **especialización**: un agente con un system prompt de 500 palabras enfocado en análisis de seguridad de código produce mejores resultados que un agente genérico con 2000 palabras que también analiza código, lo optimiza, escribe tests, y genera documentación. La especialización no es solo una cuestión de brevedad del prompt; es una cuestión de coherencia del rol. Un LLM con un rol bien definido y acotado toma decisiones más consistentes dentro de ese dominio que uno que debe balancear múltiples responsabilidades con criterios posiblemente contradictorios.

La segunda limitación es el **paralelismo**: tareas decomponibles en subtareas independientes pueden ejecutarse simultáneamente en múltiples agentes, reduciendo la latencia total de horas a minutos en workflows de procesamiento masivo. Un agente único que analiza 20 documentos en secuencia tarda 20 veces más que 20 agentes que analizan un documento cada uno en paralelo. En workflows de investigación competitiva, análisis de múltiples fuentes, o procesamiento de grandes volúmenes de datos, el paralelismo es el único mecanismo que puede satisfacer los SLOs de latencia.

La tercera limitación es la **gestión del contexto**: un agente que debe mantener el contexto completo de una tarea larga consume ventana de contexto proporcionalmente a la complejidad de la tarea. Dividir la tarea en subtareas y asignar cada una a un agente especializado significa que cada agente solo necesita el contexto de su fragmento: 2-5K tokens por agente especializado en lugar de 50K tokens para el agente único. Esto no solo reduce el costo de tokens sino que mejora la calidad del razonamiento: con menos contexto que procesar, el modelo puede focalizarse más efectivamente en la subtask específica.

La cuarta consideración —frecuentemente ignorada— es la **división del espacio de herramientas**. Cuando todas las herramientas disponibles se exponen a un único agente, el espacio de decisión de selección de herramientas crece y la ambigüedad de selección aumenta. Asignar a cada agente especializado solo las herramientas relevantes para su rol reduce esta ambigüedad: el agente de investigación tiene acceso a herramientas de búsqueda web y lectura de documentos; el agente de análisis tiene acceso a herramientas de procesamiento de datos y visualización; el agente de redacción tiene acceso a herramientas de edición y formato. Cada agente trabaja con un espacio de herramientas acotado y coherente con su rol.

El **overhead de coordinación** es el costo que debe superar cualquier beneficio del multiagente. Para tareas con menos de 3-4 pasos bien definidos y sin paralelismo natural, un agente único suele ser más eficiente que un sistema multiagente: el tiempo de comunicación entre agentes, la gestión del estado compartido, la reconciliación de resultados, y el overhead de orquestación pueden superar el beneficio de la especialización y el paralelismo. La regla práctica es medir el overhead de coordinación estimado antes de adoptar la arquitectura multiagente y verificar que el beneficio es al menos 2x el costo.

Conceptos clave

- **Especialización de roles**: prompts acotados al dominio específico del agente mejoran la coherencia y precisión de las decisiones dentro de ese dominio comparado con un agente genérico con prompts más amplios
- **Paralelismo de subtareas**: subtareas independientes ejecutándose simultáneamente en múltiples agentes con `asyncio.gather()`; reduce la latencia total a la del agente más lento (max) en lugar de la suma de todos
- **Reducción de contexto por agente**: cada agente especializado solo necesita el contexto de su subtask específica; 2-5K tokens por agente vs 50K para el agente único; mejora calidad de razonamiento y reduce costo

- **División del espacio de herramientas:** asignar a cada agente solo las herramientas relevantes para su rol; reduce ambigüedad de selección y mejora precisión de invocación de herramientas
- **Overhead de coordinación:** el costo de comunicación, gestión de estado compartido y orquestación que debe ser superado por los beneficios; tareas con menos de 3-4 pasos raramente justifican el overhead multiagente

Principio rector

El multiagente es la solución correcta cuando la tarea supera la capacidad de un agente individual en términos de context window, especialización de dominio o paralelismo de subtareas, y cuando el beneficio de esa extensión supera el costo de coordinación. No es la solución correcta por defecto para cualquier tarea compleja, ni una señal de sofisticación arquitectónica per se.

La sección siguiente examina los cuatro patrones de coordinación multiagente —secuencial, paralelo, jerárquico y de debate— con los trade-offs operativos de cada uno para guiar la elección del patrón correcto según la estructura natural del problema.

Módulo 7 – Capítulo 06 – Sección 02

Patrones de coordinación: secuencial, paralelo, jerárquico y de debate

Elegir el patrón de coordinación correcto es tan importante como elegir el framework correcto: el patrón incorrecto produce sistemas que desperdician la ventaja del multiagente (usando coordinación secuencial donde el paralelismo era posible) o que añaden complejidad sin beneficio proporcional (usando coordinación jerárquica donde la secuencia era suficiente). Los cuatro patrones fundamentales describen formas diferentes de organizar la colaboración entre agentes, cada una con un perfil de latencia, determinismo y calidad de resultado diferente.

El **patrón secuencial** (pipeline) es el más simple y predecible: el output del agente N es el input del agente N+1, y así sucesivamente hasta el resultado final. La implementación es directa: `output_1 = agent_1.run(input); output_2 = agent_2.run(output_1); ...`. La latencia total es la suma de las latencias individuales, lo que es su principal limitación. El patrón secuencial es apropiado cuando las subtarefas tienen dependencias estrictas entre sí — el agente de análisis necesita el documento que el agente de extracción produjo— y cuando la corrección de cada paso depende del resultado del anterior. En CrewAI, `Process.sequential` implementa este patrón directamente; en LangGraph, corresponde a un grafo lineal sin bifurcaciones.

El **patrón paralelo** (map-reduce) ejecuta múltiples agentes concurrentemente sobre el mismo input o sobre particiones del problema, y combina sus resultados mediante una función de merge o un agente agregador. La implementación en Python usa `asyncio.gather()` para lanzar todos los agentes simultáneamente: `results = await asyncio.gather(*[agent.run(partition) for partition in partitions])`. La latencia total es la del agente más lento (max de latencias individuales) más la del agregador, lo que puede producir reducciones de latencia de 5x-20x respecto al equivalente secuencial. El patrón paralelo requiere que las subtarefas sean verdaderamente independientes: si el resultado del agente A afecta el procesamiento del agente B, el paralelismo no es válido y debe usarse

secuencial. Aplicaciones típicas: análisis de múltiples documentos, evaluación multi-criterio de una alternativa, investigación de múltiples temas simultáneamente.

El **patrón jerárquico** (orchestrator-workers) introduce un agente orquestador que planifica, delega y coordina a agentes especializados subordinados. El orquestador recibe el objetivo de alto nivel, genera un plan de subtareas, las delega a los agentes worker correctos, monitorea sus resultados, y sintetiza la respuesta final. En AutoGen, el `GroupChatManager` actúa como orquestador; en CrewAI con `Process.hierarchical`, el manager LLM hace el mismo rol; en LangGraph, un nodo de orquestación toma estas decisiones de routing. La fortaleza de este patrón es su adaptabilidad: el orquestador puede reasignar tareas cuando un worker falla, modificar el plan cuando los resultados intermedios cambian los supuestos, y añadir subtareas no anticipadas en el plan original. Su debilidad es que el orquestador es un punto de fallo central: errores de planificación del orquestador se propagan a todos los workers.

El **patrón de debate** (agent debate / sociedad de agentes) ejecuta múltiples agentes sobre el mismo problema y los hace revisar mutuamente sus respuestas, iterando hacia un consenso. El proceso típico es: cada agente genera una respuesta independiente, luego cada agente lee las respuestas de los demás y revisa la suya si hay argumentos que la contradicen o la mejoran. Después de N rondas de debate (típicamente 2-3), se aplica voting, promedio de scores, o un LLM-judge para seleccionar la mejor respuesta o sintetizar el consenso. El beneficio empírico es consistente: el patrón de debate mejora la calidad de respuestas en 10-25% en benchmarks de razonamiento complejo comparado con cualquier agente individual. El costo es igualmente consistente: es el patrón más costoso en tokens ($k \text{ agentes} \times N \text{ rondas} \times \text{tokens por respuesta}$) y el más lento en latencia. Se justifica exclusivamente cuando la calidad es la prioridad absoluta y el costo adicional es aceptable.

Los sistemas de producción reales frecuentemente combinan patrones: un orquestador jerárquico que internamente ejecuta etapas paralelas, donde el resultado de cada etapa paralela alimenta una siguiente etapa secuencial. Un sistema de análisis de riesgo puede usar un orquestador que delega análisis de datos en paralelo a tres agentes especializados, luego un agente de síntesis que combina secuencialmente los tres análisis, y finalmente un agente de debate de dos instancias que revisa la síntesis antes de producir la recomendación final.

Aspectos técnicos

- **Patrón secuencial:** latencia total = suma de latencias individuales; implementado como cadena de calls síncronos o encadenados; apropiado cuando subtareas tienen dependencias estrictas entre sí
- **Patrón paralelo:** `asyncio.gather()` para ejecución concurrente; latencia total = $\max(\text{latencias individuales}) + \text{latencia del agregador}$; requiere independencia real entre subtareas; reducción de latencia de 5x-20x respecto al equivalente secuencial
- **Patrón jerárquico:** orquestador con capacidad de planificación, delegación, monitoreo de resultados y re-planificación cuando los supuestos cambian; el modelo del orquestador debe ser el más capaz disponible (GPT-4o, Claude 3.5 Sonnet)
- **Patrón de debate:** múltiples agentes (3-5 típico) generan respuestas independientes y se revisan mutuamente; 2-3 rondas de revisión seguidas de voting o LLM-judge; mejora calidad 10-25% a costo de $k \times N$ veces más tokens y latencia
- **Patrones híbridos:** los sistemas de producción combinan patrones; un orquestador jerárquico puede ejecutar etapas paralelas internamente y usar pipeline secuencial para el post-procesamiento; diseñar el patrón que sigue la estructura natural del problema

Para recordar

La elección del patrón de coordinación debe basarse en la estructura natural del problema: si las subtareas son independientes, usar paralelismo; si tienen dependencias estrictas, usar pipeline; si requieren expertise adaptativo y gestión de fallos de workers, usar jerarquía; si la calidad supera al costo en la función objetivo del proyecto, considerar debate. La estructura natural del problema es siempre el criterio correcto, no la sofisticación deseada del sistema.

La siguiente sección examina cómo los agentes en estos sistemas se comunican entre sí de forma efectiva: los mensajes estructurados, los handoffs y los contratos de interfaz que hacen que la coordinación multiagente sea predecible y debuggeable.

Módulo 7 – Capítulo 06 – Sección 03

Comunicación entre agentes: mensajes, handoffs y contratos de interfaz

La comunicación entre agentes es el talón de Aquiles más frecuente de los sistemas multiagente en producción. Mientras que la comunicación entre microservicios tiene décadas de patrones establecidos —REST, gRPC, mensajería asíncrona con Kafka—, la comunicación entre agentes de LLM enfrenta un desafío adicional: el receptor es un sistema probabilístico que interpreta el mensaje recibido en lenguaje natural, y una descripción ambigua o un formato inconsistente puede producir una interpretación diferente a la intención del emisor. Este problema es invisible en los demos donde el agente produce exactamente el texto que el desarrollador esperaba; se manifiesta en producción cuando el output del agente A se convierte en el input del agente B en condiciones que el desarrollador no anticipó durante el desarrollo.

La solución fundamental es tratar la comunicación entre agentes con el mismo rigor formal que la comunicación entre servicios: **contratos de interfaz** que especifican exactamente el formato de input y output de cada agente. En Python, esto se implementa con modelos Pydantic que definen la estructura de los mensajes que cada agente produce y consume. Un agente de análisis de sentimientos que recibe `TextAnalysisRequest` (texto a analizar, idioma, dominio) y produce `SentimentAnalysisResult` (score de sentimiento, entidades mencionadas, resumen) tiene una interfaz tan bien definida como una función de una librería pública. Pydantic AI hace esto nativo; en LangGraph y AutoGen requiere disciplina de diseño del equipo.

Los **mensajes estructurados** son la unidad de comunicación entre agentes. Un mensaje bien diseñado para sistemas multiagente incluye: el resultado principal del procesamiento del agente emisor, metadata de ejecución (`agent_id`, `task_id`, `timestamp`), un indicador de confianza (`confidence_score` entre 0 y 1), cualquier error parcial o advertencia (`warnings`), y estadísticas de consumo de recursos (`tokens_used`, `latency_ms`). Esta metadata permite al agente receptor o al orquestador tomar decisiones informadas: si el `confidence_score` es menor de 0.6, quizás el resultado debe ser revisado por otro agente antes de usarse; si

`tokens_used` supera un umbral, quizás la subtask fue más compleja de lo esperado y el plan debe ajustarse.

El **patrón de handoff** de Anthropic es el mecanismo más documentado para la transferencia de control entre agentes con contexto completo. En el flujo de Claude, un agente puede devolver una herramienta `handoff_to_agent(agent_id, context)` que transfiere el control al agente receptor junto con el historial de conversación completo hasta ese punto. El agente receptor no empieza desde cero: hereda el contexto acumulado del agente emisor, incluyendo las observaciones recopiladas, las decisiones tomadas, y el progreso hacia el objetivo. Este patrón garantiza continuidad de contexto pero puede producir contextos muy largos si el historial acumulado antes del handoff es extensivo.

El debate entre **message passing** y **shared state** como paradigmas de comunicación refleja un trade-off fundamental. Message passing —cada agente produce mensajes explícitos que se enrutan a los receptores apropiados— es más desacoplado y escalable: los agentes son independientes entre sí y pueden evolucionar sin afectar a los demás mientras respeten los contratos de interfaz. Shared state —un estado global accesible por todos los agentes, como el `StateGraph` de LangGraph— es más conveniente para agentes que colaboran estrechamente sobre el mismo problema: cualquier agente puede leer el estado actual completo sin esperar mensajes explícitos. Los sistemas de producción complejos frecuentemente combinan ambos: estado compartido para el contexto de la tarea actual, y mensajes explícitos para la transferencia de resultados entre etapas.

La **serialización** de los mensajes es un requisito operativo que se ignora en desarrollo local y produce bugs en producción. Los mensajes entre agentes que cruzan boundaries de proceso — diferentes workers, diferentes servicios— deben ser serializables a JSON: nunca usar `pickle` para serializar mensajes inter-agente porque `pickle` es inseguro (puede ejecutar código arbitrario durante la deserialización), no es compatible entre versiones de Python, y no es legible para debugging. Los modelos Pydantic se serializan trivialmente a JSON con `.model_dump_json()` y se deserializan con `Model.model_validate_json()`.

Aspectos técnicos

- **Contratos de interfaz (Pydantic schemas):** definir modelos de input y output de cada agente como clases Pydantic; validar el input antes de enviarlo y el output antes de procesarlo; la interfaz del agente es tan explícita como la de una función pública en una librería

- **Mensajes estructurados con metadata:** incluir resultado principal + `agent_id`, `task_id`, `confidence_score`, `warnings`, `tokens_used`; la metadata permite al orquestador tomar decisiones informadas sobre cómo proceder con el resultado
- **Handoff pattern:** transferencia de control entre agentes con el historial de contexto completo acumulado; garantiza continuidad pero puede producir contextos extensos si el historial previo al handoff es largo
- **Message passing vs shared state:** message passing para agentes desacoplados que pueden evolucionar independientemente; shared state (LangGraph StateGraph) para agentes que colaboran estrechamente y necesitan acceso al estado completo de la tarea
- **Serialización JSON:** todos los mensajes que cruzan boundaries de proceso deben ser JSON-serializable; nunca usar pickle; Pydantic provee serialización/deserialización JSON nativa con validación de tipos

Principio rector

Los contratos de interfaz entre agentes son tan importantes como los contratos de API entre microservicios: la informalidad en la comunicación inter-agente produce bugs que son difíciles de atribuir a un agente específico porque el error es una propiedad emergente de la interacción, no de ningún agente individual.

La sección siguiente examina las responsabilidades específicas del agente orquestador y de los agentes especializados, con los criterios para mantener esas responsabilidades bien delimitadas en sistemas que evolucionan.

Módulo 7 – Capítulo 06 – Sección 04

Agente orquestador vs agentes especializados: roles y responsabilidades

En sistemas multiagente con patrón jerárquico, la delimitación clara entre el agente orquestador y los agentes especializados es la diferencia entre un sistema cohesivo y uno donde la responsabilidad de cada componente es ambigua. Los sistemas donde el orquestador ejecuta tareas operativas que debería delegar, o donde los agentes especializados intentan planificar el workflow global que no les corresponde, producen comportamientos ineficientes, difíciles de debuggear, y propensos a conflictos cuando sus objetivos divergen. La separación de responsabilidades en sistemas multiagente no es solo un principio de diseño; es un requisito de mantenibilidad.

El **agente orquestador** es el estratega del sistema. Sus responsabilidades son exclusivamente de nivel superior: recibir el objetivo de alto nivel y descomponerlo en subtareables ejecutables, decidir qué agente especializado ejecuta qué subtarea y en qué orden o en paralelo, monitorear los resultados intermedios de los workers para verificar que se alinean con los supuestos del plan, re-planificar cuando esos supuestos no se cumplen, y sintetizar los outputs de los workers en un resultado final coherente. El orquestador no ejecuta operaciones del dominio directamente: no hace búsquedas web, no ejecuta código, no lee documentos. Si lo hace, está asumiendo responsabilidades de los workers y convirtiéndose en un cuello de botella.

El modelo de LLM del orquestador merece atención específica. El orquestador requiere el modelo con mejor capacidad de razonamiento y planificación disponible —GPT-4o, Claude 3.5 Sonnet— porque sus errores de planificación se propagan a todos los agentes subordinados. Un error del orquestador en el paso de delegación afecta el resultado de todos los workers que ejecutan basándose en ese plan incorrecto. Los workers, por contraste, ejecutan subtareables acotadas y bien definidas donde un modelo más ligero y rápido puede ser suficiente: un modelo que es excelente para tareas de búsqueda y síntesis de texto puede funcionar perfectamente como worker de investigación, incluso si no es el modelo más capaz para tareas de planificación estratégica.

Los **agentes especializados** (workers) son los ejecutores expertos en su dominio. Sus responsabilidades son exclusivamente operativas: recibir la subtask asignada por el orquestador, ejecutarla usando las herramientas disponibles en su toolkit, producir el entregable definido en el contrato de interfaz, e informar al orquestador si la subtask produjo resultados inesperados que podrían requerir re-planificación. Los workers idealmente son **stateless entre invocaciones**: todo el contexto necesario para ejecutar la subtask se les provee en cada llamada, y su output se devuelve al orquestador sin mantener estado propio. La statefulness de los workers complica el escalado horizontal (cada instancia del worker tiene estado diferente) y la recuperación ante fallos (si el worker crashea, su estado se pierde).

La delimitación entre orquestador y worker no siempre es obvia en el diseño inicial y tiende a degradarse a medida que el sistema evoluciona. Un orquestador que empieza como puro planificador puede ir acumulando lógica operativa cuando el equipo añade "pequeñas excepciones" que parecen más simples de manejar en el orquestador que de delegar a los workers. El síntoma de esta degradación es un orquestador cuyo system prompt crece indefinidamente y cuyo comportamiento se vuelve difícil de predecir. La revisión periódica del system prompt del orquestador —eliminando cualquier lógica que debería estar en un worker especializado— es una práctica de mantenimiento que vale la pena establecer desde el inicio del proyecto.

La **delegación vs ejecución directa** es una decisión que el orquestador debe poder tomar: para subtasks triviales que no justifican el overhead de comunicación con un worker (una verificación simple, una transformación de formato), el orquestador puede ejecutarlas directamente sin delegación. La regla práctica es: si la subtask puede completarse en un solo paso de razonamiento sin herramientas adicionales, el orquestador puede hacerlo directamente; si requiere herramientas o múltiples pasos de ejecución, debe delegarse al worker apropiado.

Aspectos técnicos

- **Responsabilidades del orquestador:** planificación (descomposición del objetivo en subtasks), delegación (asignación de subtasks a workers), monitoreo de resultados intermedios, re-planificación cuando los supuestos cambian, síntesis del resultado final; el orquestador nunca ejecuta operaciones del dominio directamente
- **Modelo de LLM para el orquestador:** el más capaz disponible (GPT-4o, Claude 3.5 Sonnet); sus errores de planificación se propagan a todos los workers; el costo adicional de

un modelo más capaz para el orquestador se justifica por el impacto sistémico de sus decisiones

- **Workers stateless:** los agentes especializados reciben todo el contexto necesario en cada invocación y devuelven el resultado sin mantener estado propio; facilita el escalado horizontal y la recuperación ante fallos
- **División de responsabilidades por sistema prompt:** el system prompt del orquestador no debe incluir lógica operativa del dominio; el system prompt de cada worker no debe incluir lógica de planificación global; revisar periódicamente para detectar degradación de esta separación
- **Delegación vs ejecución directa:** el orquestador puede ejecutar directamente subtareas que no requieren herramientas ni múltiples pasos; delegar cuando la subtask requiere herramientas o ejecución en el dominio especializado del worker

Para recordar

El orquestador es el estratega y coordinador del sistema; los agentes especializados son los ejecutores expertos en su dominio. Confundir estos roles —un orquestador que ejecuta tareas operativas del dominio, o un especialista que intenta planificar el workflow global— produce sistemas ineficientes donde nadie tiene una responsabilidad clara y los errores son difíciles de atribuir y corregir.

La sección siguiente examina qué ocurre cuando la coordinación entre agentes produce resultados contradictorios, y los mecanismos de reconciliación que permiten al sistema resolver esas contradicciones de forma robusta.

Módulo 7 – Capítulo 06 – Sección 05

Consistencia y reconciliación: qué sucede cuando dos agentes producen resultados contradictorios

La inconsistencia entre agentes no es un edge case improbable en sistemas multiagente: es una consecuencia esperada y estadísticamente frecuente cuando múltiples agentes evalúan el mismo problema desde perspectivas especializadas diferentes. Un agente de análisis de seguridad clasifica un fragmento de código como vulnerable; el agente de análisis de rendimiento lo clasifica como óptimo y no recomienda cambios. Un agente de marketing recomienda aumentar el precio; el agente de análisis competitivo recomienda reducirlo. Estas contradicciones no son errores de los agentes —cada uno puede estar correctamente razonando desde su perspectiva especializada—; son el resultado de que diferentes dimensiones de análisis producen recomendaciones que entran en conflicto en el nivel de la decisión final.

El problema es que muchos sistemas multiagente no están diseñados para manejar estas contradicciones. Sin un mecanismo explícito de reconciliación, el orquestador se enfrenta a tres opciones igualmente problemáticas: elegir arbitrariamente uno de los dos resultados contradictorios sin racionalidad explícita, propagar la contradicción al output final y dejar que el usuario la resuelva, o quedar bloqueado sin saber cómo proceder. Ninguna de estas opciones es aceptable en un sistema de producción confiable. El mecanismo de reconciliación debe ser diseñado antes de que aparezcan las primeras contradicciones, no como reacción a ellas.

La primera tarea es la **detección de contradicciones**. El orquestador debe verificar activamente si los outputs de los agentes son consistentes entre sí antes de sintetizarlos. Para outputs categóricos (clasificaciones, recomendaciones binarias), la detección es directa: ¿los agentes llegaron a la misma categoría? Para outputs semánticos (análisis en texto libre), las métricas de detección incluyen: la similaridad coseno entre embeddings de las respuestas (valores menores de 0.7 indican divergencia semántica significativa), análisis de inferencia natural (NLI) para detectar contradicción explícita entre pares de respuestas, o verificación programática para outputs estructurados con campos comparables. La detección

automatizada permite al orquestador aplicar la política de reconciliación apropiada sin depender del razonamiento del LLM para identificar el conflicto.

Los **mecanismos de reconciliación** tienen diferentes trade-offs de calidad, costo y velocidad. El **voting por mayoría** es el más simple: para k agentes que producen outputs categóricos, la respuesta mayoritaria gana; efectivo para $k \geq 3$ impar, donde hay garantía de mayoría. El **promedio de scores** es apropiado para outputs numéricos como scores de confianza, estimaciones de tiempo, o ratings de calidad; el orquestador puede ponderar los agentes según su autoridad sobre el aspecto específico en conflicto. El **debate estructurado** es el mecanismo de mayor calidad pero más costoso: presentar a cada agente el argumento del otro con una instrucción de revisión ("El agente B argumenta X basándose en evidencia Y; ¿cambia esto tu evaluación?"), iterar 1-2 rondas, y usar un LLM-judge para determinar qué posición está mejor fundamentada. El **escalado al humano** es la opción correcta cuando la divergencia supera un umbral predefinido y cuando la decisión tiene alto impacto: dos agentes con outputs diametralmente opuestos y alta confianza en ambos casos es precisamente la situación donde el juicio humano es irremplazable.

La **ponderación de agentes** por dominio de autoridad es un refinamiento importante del voting simple. No todos los agentes tienen la misma autoridad sobre todos los aspectos de un problema: el agente de seguridad es más autoritativo sobre las implicaciones de seguridad de una decisión, mientras que el agente de rendimiento es más autoritativo sobre las implicaciones de performance. Un sistema que pondera los votos de cada agente según su dominio de expertise produce reconciliaciones más alineadas con el razonamiento correcto que el voting uniforme.

Puntos críticos

- **Detección automatizada de contradicciones:** antes de sintetizar resultados de múltiples agentes, verificar consistencia; similaridad coseno < 0.7 entre embeddings de respuestas, NLI para contradicción semántica, verificación programática para outputs estructurados con campos comparables
- **Voting por mayoría:** para $k \geq 3$ agentes con outputs categóricos; efectivo cuando todos los agentes evalúan el mismo aspecto con criterios comparables
- **Promedio o mediana de scores:** para outputs numéricos (confianza, estimaciones, ratings); ponderar agentes según su autoridad sobre el aspecto específico en conflicto
- **Debate estructurado:** presentar el argumento contrario a cada agente para revisión; 1-2 rondas de debate + LLM-judge; mayor calidad de reconciliación al costo de tokens

adicionales por cada round

- **Escalado al humano por threshold:** cuando la divergencia supera un umbral predefinido (dos agentes con outputs opuestos y alta confianza), escalar a revisión humana en lugar de resolver artificialmente

***Nota del Arquitecto:** El mecanismo de reconciliación que más frecuentemente se omite en el diseño inicial y más se lamenta en producción es el escalado al humano por threshold. Es tentador diseñar sistemas que siempre producen una respuesta, porque un sistema que "se bloquea" pide intervención humana parece menos capaz. En realidad, un sistema que escala las contradicciones genuinas al humano en lugar de resolverlas arbitrariamente es más confiable, no menos. Diseñar el threshold de escalado desde el primer día.*

Para recordar

La reconciliación de resultados contradictorios no es un edge case sino un requisito de diseño en cualquier sistema multiagente donde múltiples agentes evalúan el mismo aspecto de un problema. El mecanismo debe ser diseñado explícitamente antes de que aparezcan las primeras contradicciones, porque las contradicciones son una consecuencia esperada de la especialización, no un bug del sistema.

La sección siguiente cierra el capítulo multiagente con la perspectiva que equilibra el potencial de estos sistemas con la complejidad operativa que inevitablemente introducen.

Módulo 7 – Capítulo 06 – Sección 06

Cierre: los sistemas multiagente amplifican tanto la capacidad como la complejidad

El capítulo sobre sistemas multiagente establece que la arquitectura multiagente es una decisión de ingeniería con consecuencias dobles: amplifica la capacidad del sistema para manejar tareas complejas mediante especialización y paralelismo, pero también amplifica la complejidad operativa, los puntos de fallo potenciales, el costo de tokens, y la dificultad de debugging. Un sistema de 5 agentes no es 5 veces más poderoso que un agente individual; es potencialmente más poderoso en el dominio del problema que resuelve, pero también significativamente más complejo en términos de observabilidad, testing, gestión de estado compartido, y reconciliación de resultados contradictorios.

Las tres motivaciones legítimas para adoptar la arquitectura multiagente —especialización de dominio, paralelismo de subtarefas, reducción del contexto por agente— deben estar presentes como limitaciones concretas y medibles del agente individual antes de justificar el overhead de un sistema multiagente. Un equipo que adopta multiagente porque "suena más sofisticado" o porque "el agente único ya es complejo" está añadiendo complejidad sin beneficio proporcional: los mismos problemas que hacen complejo al agente individual se vuelven más difíciles de diagnosticar en un sistema de múltiples agentes donde los errores pueden originarse en cualquier nodo del grafo de coordinación.

La inversión que debe acompañar la adopción del multiagente es proporcional a su complejidad. Un sistema de 5 agentes requiere: testing de integración entre los agentes (no solo unit tests de cada agente individualmente), observabilidad que capture la traza completa de una tarea a través de todos los agentes (LangSmith o Langfuse con trazas anidadas), políticas de manejo de inconsistencias entre agentes, y procedimientos de respuesta a incidentes que identifiquen en qué agente específico se originó un fallo. Sin estas inversiones, el sistema multiagente produce un tipo de complejidad operativa que es más difícil de manejar que la complejidad de un agente único bien diseñado.

El capítulo siguiente —sobre testing de agentes— aplica estas consideraciones a la pregunta de cómo verificar que el comportamiento de estos sistemas, sean de un agente único o multiagente, cumple los criterios de calidad antes de desplegarse en producción. El testing de agentes es el puente entre el diseño y el despliegue seguro.

Para recordar

El multiagente es la solución correcta cuando las limitaciones del agente individual son el cuello de botella medible de un sistema. Es la complejidad incorrecta cuando se adopta como aspiración arquitectónica antes de que esas limitaciones sean evidentes y cuantificadas. Medir primero, escalar después.

"Simplicity is the ultimate sophistication." — atribuido a Leonardo da Vinci; en el contexto de sistemas multiagente, la arquitectura más simple que resuelve el problema es siempre preferible a la más sofisticada que también lo resuelve. Cada agente adicional en un sistema debe justificarse con un beneficio medible que supere su costo en complejidad operativa.